Appendix A. Java Then and Now

Introduction: Always in Motion the Java Is

Java has always been a moving target for developers and writers. This I know as I've been updating, first a Java training program and later this book, for pretty much all of Java's lifetime. Some developers I meet in my commercial training programs are still not aware of some of the features added to ancient Java releases, let alone current ones. This appendix offers a look at each recent release of Java. Details on releases prior to Java 16 are considered ancient history and have been moved to my website, *https://darwinsys.com/java/ancientHistory.html*. For a review of the very early history, see Jon Byous's Sun Microsystems article "Java Technology: The Early Years".

There are many references to "JEP" (Java Enhancement Proposal) in this section. To get more information on any of these, simply visit *openjdk.org/jeps/NNN* where *NNN* is the JEP number.

What Was New in Java 16 16

The biggest items in Java are the `record` type and the `jpackage` tool, but there are many more.

Another big change, not in the language but in how the JDK source code is maintained, is the move from half a dozen separate repositories in Mercurial, a smaller competitor to Git, into a unified repository on GitHub. These changes were introduced in JEP 357: Migrate from Mercurial to Git and JEP 369: Migrate to GitHub, and I wrote about them in Oracle's *Java Magazine*.

Other changes of note include supported ports of the JDK for JEP 386: Alpine Linux (one of the smallest mainstream Linux distros, often used in cloud deployments) and JEP 388: Windows on AArch64 (64-bit ARM processors).

Java 16 Language Changes

The biggest single change is probably the addition of the `record` type (JEP 395, previously in preview, and covered in Recipe 8.4). The `record` type allows the rapid production of an immutable data holder object, with all getters and ancilliary methods provided.

Another change is pattern matching with `instanceof` (JEP 394). This allows you to replace:

```
if (x instance of MyClass) {
        MyClass m = (MyClass)x;
        // do something with m
}
```

with just:

```
if (x instance of MyClass m) {
        // do something with m, no cast needed!
}
```

There is more on pattern matching in later releases of Java, as this is just the first step toward a language feature called pattern matching.

JEP 396 marks the primitive wrapper classes as value-based and deprecates their constructors (only) for removal. It also warns about improper attempts to synchronize (see Recipe 11.4) on an instance of any JDK value-based class. (There is a private annotation `jdk.internal.ValueBased` used internally and recognized by `javac`.)

Sealed classes (JEP 397, see Recipe 8.11) continued as a preview feature in Java 16.

Java 16 API Changes

You can simply collect a `Stream` into a `List`: the terminal operation `collect(Collectors.toList())` can be replaced with just `toList()`. Be aware that this does produce an immutable list.

JEP 396 tightens the encapsulation of JDK internal classes to include all but a very few, such as the notorious-but-necessary `Unsafe` class.

Incubation features include the Vector API (JEP 338, covered in Recipe 5.11), the Foreign Linker API, and the Foreign Function and Memory (FFI/FFM) API.

Unix domain sockets JEP 380, a facility in Unix since 1980, was added to Microsoft Windows, allowing for it to be supported in Java 16. See Recipe 15.8.

One format character is added to `DateTimeFormatter` (Recipe 6.2): `B` allows a more verbose period instead of just a.m. or p.m., such as "in the afternoon."

Java 16 JDK Tools Changes

The `jpackage` tool (JEP 392: Packaging Tool, covered in Recipe 2.18) provides a means of packaging a Java application along with all the APIs that it needs *and* the subset of the JDK that is needed to run it. Works on Windows, Linux, and macOS.

Java 16 JVM Changes

Improvements to garbage collection (GC) include the release of class metadata memory and the JEP 439: Generational ZGC.

What Was New in Java 17 LTS 17

Java 17 was the next long-term supported release and incorporated several useful features.

Java 17 Language Changes

JEP 409 Sealed classes (Recipe 8.11) move from preview to final.

Java 17 API Changes

The random number functionality is significantly expanded, as per JEP 356. I've written this up in a two-part series in Oracle's *Java Magazine*.

Java 17 switches to strict floating-point calculations by default (JEP 306), so the keyword `strictfp` is now optional and ignored; all FP calculations are done in strict conformance to the IEEE 754 floating-point standard.

Better rendering on macOS (JEP 382), and initial support for macOS AArch64 (Apple's ARM 64 M1, M2, ... CPUs) are in JEP 391.

JEP 406 introduces pattern matching for `switch` in preview—as mentioned earlier, pattern matching is coming to several parts of Java syntax.

JEP 412 Foreign Function and Memory API (Incubator) to allow access to native code and memory. See Recipe 18.5.

JEP 414 Vector API (Second Incubator)—for faster numerical operations on arrays. See Recipe 5.11.

JEP 403 Strongly Encapsulate JDK Internals—makes it even harder to access JDK internal APIs that you shouldn't be accessing anyway :-)

The obsolete Security Manager (JEP 411) and the long-dead Applet API (JEP 398) are deprecated for removal—both are done and it's time for them to go.

The Experimental AOT and JIT compilers were removed in JEP 410.

The Activation mechanism of RMI was removed in JEP 407.

What Was New in Java 18 18

`finalize()` finally finding its fate, and a free web server, are two of the biggest changes.

Java 18 Language Changes

The `finalize()` method has finally been deprecated for removal in JEP 421. It was error-prone from the beginning of time, and experts have advised against its use since forever. It will be gone soon, and that's `final`.

Java 18 API Changes

The default charset is now UTF-8 on all platforms thanks to JEP 400, allowing for better file interchange between Java on different operating systems.

A simple web server for static files has finally been added via JEP 408 (Python has had such a thing for a dog's age). This is invoked via the `jwebserver` command-line tool. It

can also be started using this *unsupported* incantation; don't blame me if this changes in a future release, though I've tested it up to Java 24:

```
java --add-modules=jdk.httpserver sun.net.httpserver.simpleserver.JWebServer
```

In JEP 416, some core parts of the Reflection API (Chapter 17) have been reimplemented on top of the `MethodHandles` API (Recipe 17.5) to reduce code duplication inside the JDK.

Pattern matching for `switch` is in its second preview (JEP 420).

The Vector API (Recipe 5.11) is in its third incubator.

The Foreign Function and Memory API (Recipe 18.5) is in its second incubator.

JEP 418 provides a service-provider interface (SPI) for DNS host resolution, so the `java.net.InetAddress` class can use resolvers other than the platform's built-in resolver.

Java 18 Tooling Changes

In JEP 413, Javadoc gains the ability to include code snippets (see "Javadoc Snippets"), meant to illustrate how to use a given class.

What Was New in Java 19 19

Java 19 continues the steady evolution of Java.

Java 19 Language Changes

Record patterns in preview (JEP 405) allows nesting of record patterns and type patterns. This extends pattern matching to allow for more sophisticated and composable data queries.

Pattern matching for `switch` (JEP 427), in its third preview, allows pattern matching in `switch` expressions and statements, permitting an expression to be checked against multiple patterns.

These two together are significant steps toward what's being called data-oriented programming (see Recipe 8.15).

Java 19 API Changes

Foreign Function and Memory API (Recipe 18.5) moves from incubator to preview (JEP 424).

The Vector API (Recipe 5.11) is back, now in its fourth incubator (JEP 426).

Virtual threads (Recipe 11.2) appears in its first preview (JEP 425).

Structured concurrency (JEP 428, see Recipe 11.7) is here as an incubator feature.

Java 19 JVM Changes

A port to the RISC/V processor on Linux, via JEP 422.

What Was New in Java 20 20

Nothing radically new here—all but one of the major enhancements are continued preview or incubator features.

Java 20 Language Changes

Record patterns appears in its second preview (JEP 432).

Pattern matching for `switch` in its fourth preview (JEP 433).

Java 20 API Changes

Scoped values (JEP 429, described in Recipe 11.7) is new, as an incubator feature.

Virtual threads (JEP 436, covered in Recipe 11.2) is in its second preview.

Structured concurrency (JEP 437, see Recipe 11.7) is in its second incubator.

Foreign Function and Memory API (JEP 434, described in Recipe 18.5) continues as a second preview.

The Vector API (JEP 438, covered in Recipe 5.11) is in its fifth incubator release.

What Was New in Java 21 LTS 21

Java 21 is the latest and greatest LTS as of this writing, but, of course, not for long, as new releases appear every six months. It will be the current LTS release until, we believe, 2025.

Java 21 Language Changes

Record patterns in `switch` (JEP 440) and pattern matching for `switch` (JEP 441) together have been modified to allow for simpler use of `switch` statements and record unpacking; see Recipe 8.9.

JEP 443 in preview offers unnamed patterns and variables—using `_` for the name of an argument that is never going to be used.

Java 21 API Changes

Virtual threads (JEP 444) allow for significant performance improvement of non-CPU-bound applications (see Recipe 11.2).

Sequenced collections (JEP 431) allow navigation of collections with first, last, and next element retrieval.

JEP 452 offers a new Key Encapsulation Mechanism API (KEMs can be used in cryptography to pass keys securely).

Java 21 JVM Changes

Generational ZGC (JEP 439)—the ZGC garbage collector now uses both an old and a new space, which aims to improve performance beyond what ZGC brought previously.

JEP 449 deprecates the Windows 32-bit x86 port for removal.

JEP 451 prepares to to disallow dynamic loading of monitoring agents.

Java 21 Preview Changes

One of the biggest changes is unnamed classes and instance main (JEP 445, see Recipe 1.2).

Another biggie was the ill-fated string interpolation (JEP 430).

Structured concurrency (JEP 453) and scoped values (JEP 446) provide better support for concurrent applications.

The Foreign Function and Memory API (JEP 442) allows for easier access to native code and the ability to share memory with a native function.

Java 21 Incubator Changes

The Vector API (JEP 448, see Recipe 5.11) has minor improvements.

What Was New in Java 22 22

Java 22 Language Changes

JEP 456, Unnamed Variables & Patterns, now out of preview, allows use of `_` as a variable name in many contexts to signify that the variable is not used, as in this Swing button handler:

```
quitButton.addActionListener(_ -> System.exit(1));
```

The argument (which is of type `ActionListener`) would normally have been declared with a name like `e` or `evt`, but is not actually used in the lambda; the `_` optimization tells the compiler explicitly to disregard it.

Java 22 API Changes

JEP 454, the Foreign Function and Memory API, is out of preview (see Recipe 18.5).

JEP 457 provides a preview of the Class-File API (see Recipe 17.11).

Java 22 Tooling Changes

`javac` is extended by JEP 458 to allow you to launch multi-file source-code programs.

Java 22 Preview Changes

Statements before `super(...)` (JEP 447) allows statements that don't reference current-object fields to appear before a call to `super()`, allowing simplification of error checking, and otherwise relaxes the strictness; see Recipe 8.2.

JEP 461: Stream Gatherers enhances the Streams API (Recipe 9.3) to provide for easier coding of intermediate operators. Covered in Recipe 9.5.

Following remain in preview:

- JEP 459: String Templates or String Interpolation remained a preview feature in Java 22 (but was sadly killed off in Java 23).
- JEP 462: Structured Concurrency (Second Preview)
- JEP 463: Implicitly Declared Classes and Instance Main Methods (Second Preview)
- JEP 464: Scoped Values (Second Preview)

Java 22 Incubator Changes

The Vector API (Recipe 5.11) reaches a record-setting seventh incubation, with only minor changes.

What's New in Java 23 23

This is the latest release as this book is going to press for its fifth edition.

Java 23 language Changes

JEP 471 deprecates for removal the memory-access methods in `sun.misc.Unsafe`.

Implicitly declared classes and instance main methods (JEP 477) are in their third preview.

JEP 455 offers the initial preview of primitive types in patterns, `instanceof`, and `switch`.

Module import declarations (JEP 476, in preview) allow you to import all the exports from a given module with one statement: `import module` *`modulename`*.

Java 23 API Changes

The Vector API is in its eighth(!) incubation, as JEP 469.

Stream gatherers (JEP 473) and the Class-File API (JEP 466), both in their second preview.

Java 23 JVM Changes

JEP 474 makes the generational mode of the ZGC garbage collector the default mode.

Java 23 Tooling Changes

JEP 467: Javadoc now accepts Markdown documentation comments; see "Markdown 23" for an example.

Java 23 Preview

String templates preview is *removed*, which I don't think has happened before, or at least very rarely.

The following remain in preview:

- JEP 480: Structured Concurrency (Third Preview)
- JEP 481: Scoped Values (Third Preview)
- JEP 482: Flexible Constructor Bodies (Second Preview)

What's New in Java 24 24

Java 24 will be released within a week or so of this book's release. The following JEPs have been announced for it, and this is claimed to be the final list.

Language Changes

- JEP 472: Prepare to Restrict the Use of JNI (not to deprecate, but to require explicit enabling of risky native code)
- JEP 488: Primitive Types in Patterns, `instanceof`, and `switch` (Second Preview)
- JEP 492: Flexible Constructor Bodies (Third Preview)
- JEP 494: Module Import Declarations (Second Preview)

- JEP 495: Simple Source Files and Instance Main Methods (Fourth Preview)

API Changes

- JEP 478: Key Derivation Function API (Preview)
- JEP 484: Class-File API (out of preview)
- JEP 485: Stream Gatherers (out of preview)
- JEP 486: Permanently Disable the Security Manager
- JEP 487: Scoped Values (Fourth Preview)
- JEP 489: Vector API (in a record-setting ninth incubator)
- JEP 498: Warn upon Use of Memory-Access Methods in sun.misc.Unsafe
- JEP 499: Structured Concurrency (Fourth Preview)
- JEP 496: Quantum-Resistant Module-Lattice-Based Key Encapsulation Mechanism
- JEP 497: Quantum-Resistant Module-Lattice-Based Digital Signature Algorithm

GC Changes

- JEP 404: Generational Shenandoah (Experimental)
- JEP 475: Late Barrier Expansion for G1 Garbage Collection
- JEP 490: ZGC: Remove the Non-Generational Mode

JVM Changes

- JEP 450: Compact Object Headers (Experimental)
- JEP 479: Remove the Windows 32-bit x86 Port
- JEP 483: Ahead-of-Time Class Loading & Linking
- JEP 491: Synchronize Virtual Threads without Pinning
- JEP 493: Linking Run-Time Images without JMODs
- JEP 501: Deprecate the 32-bit x86 Port for Removal

Looking Ahead

> *Nobody's armchair is a good predicter of the future.*
>
> —Mike O'Dell

One thing we know is that there will be a new release of Java every six months, off into the indeterminate and unknowable future. I hope that string templates will come back to

life, and that most of the other preview and incubator features in recent releases will become standard.

Previous chapter
< Afterword

Next chapter
Index >